

AutoArchInstaller

About AutoArchInstaller

This project consists of a set of initial commands, that must be typed one by one, and three installation scripts that run the necessary commands to automate a complete Arch Linux installation. I have spent a large amount of time searching for and also deciding which were the best programs to perform different tasks according to my needs and I have automated their installation with the scripts.

Each user is different, some are looking for extreme efficiency, others for a nice-looking interface, privacy-focused programs, etc. Therefore, maybe your needs are different than mine, and to fit them you would need to install other programs. In that case, feel free to modify the scripts to install the most suitable software for you.

I have always been attracted by the GNU/Linux operating system and the Free and Open-Source Software in general because of several reasons:

- Better and more secure design than other operating systems in the marketplace.
- Open-source and more secure software since the code is continuously being analyzed by the community.
- The possibility of automating many different kind of tasks.
- Know exactly what my operating system is doing internally.
- "Free" of spyware. This is not always the case since some distributions such as Fedora have recently introduced built-in telemetry.
- Less malware.
- Many development and hacking tools available.
- Highly customizable options.

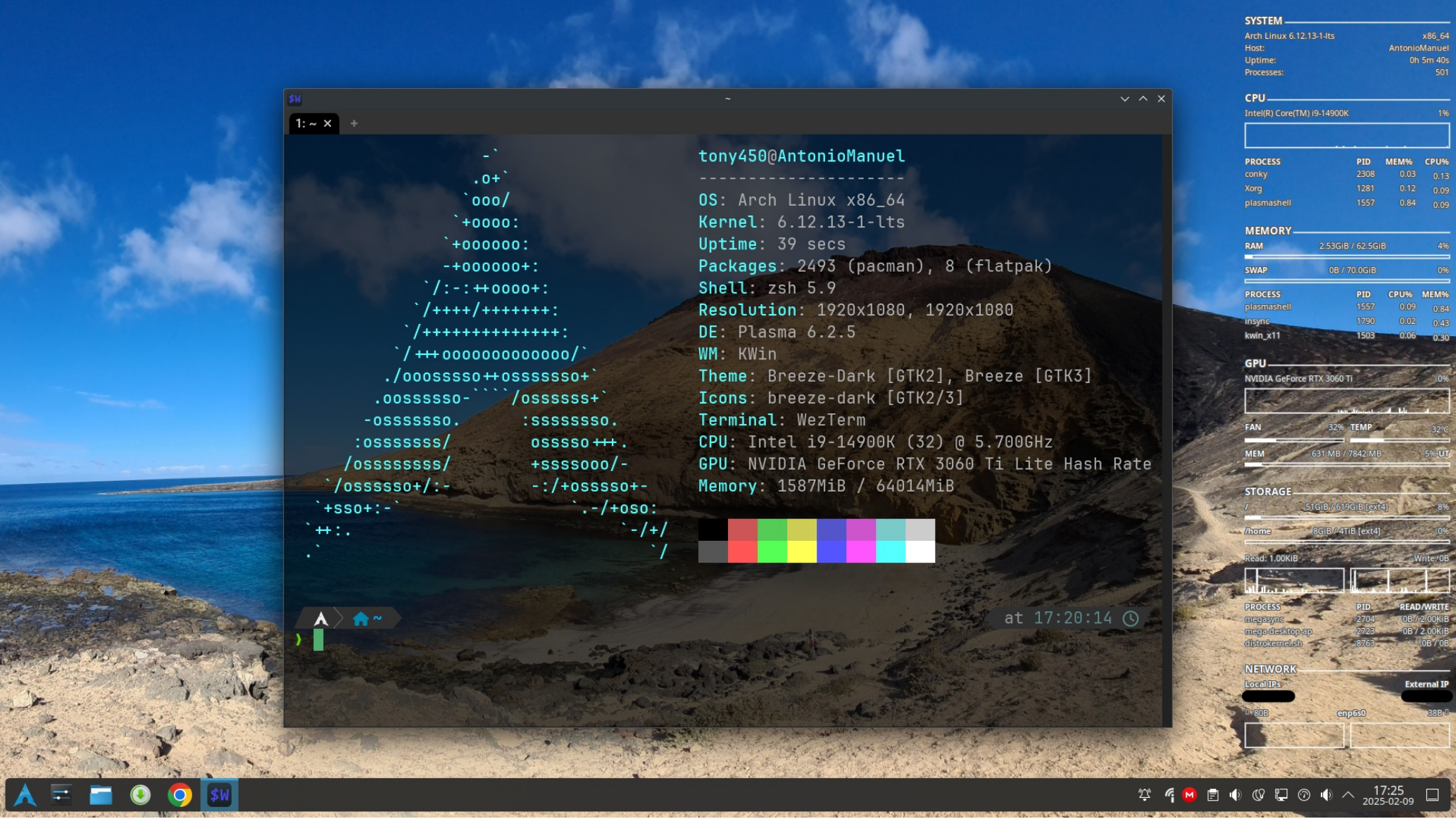
However, what I never liked about the GNU/Linux operating system, when I tried the first typical distributions (Ubuntu, Mint and Kali) was the interface. In my humble opinion, the community hasn't worked enough on this aspect, and Windows and Mac operating systems have this feature more developed. Then, I investigated how to customize the operating system in a way that fitted my needs. Firstly, I started searching for the best Desktop Environments to personalize an already built GNU/Linux distribution that I would choose such as Ubuntu or Kali, and after some time I discovered the Arch distribution.

What firstly caught my eye was that this OS is not like the other ones that I ever tried. When you first turn your machine on and enter into the intallation media, a terminal prompt is shown waiting for commands to be typed. That means you have the freedom to install each single software that you want to use on your computer from the very first moment. Additionally, the rolling-release update method is also a brilliant aspect, not only from a new features perspective but also from a security perspective, because it allows you to upgrade each package to the last version typing one single command. This is something that in other distros such as Debian or Ubuntu is not present.

I am a Computer Science Engineer specialized in Cybersecurity and the Arch Linux operating system is simply perfect for me. Therefore, I decided to automate as much as possible its installation to have the possibility of installing it quickly on new computers or on virtual machines for testing and learning purposes. If you are a curious person like me and want to know more about GNU/Linux or how an operating system works in general, then I encourage you to run the commands of the scripts and also customize them. I have learned a lot and the process has been pretty fun.

The websites from which I started reading about Arch Linux were [The Arch Linux Wiki](#) and [The Arch Linux Handbook - Learn Arch Linux for Beginners](#). They are extremely useful and many of the first set of commands and the commands of the base-install script are very similar to the ones published there.

The image below shows the final appearance of my customized Arch Linux installation.



I hope that you find this project useful and also that you enjoy customizing the installation process as much as I have.

NOTES:

- Make sure UEFI mode is enabled.
- `wlanX` , `wifi_name` and `/dev/sdX` are just generic names in order not to confuse you. Use the correct names depending on your hardware.
- It is recommended to use the Long Term Support (LTS) kernel version instead of the Stable one. Sometimes, I have experienced issues after upgrading with some applications such as VMware because of the kernel version.
- Use X11 and bridge network connection options for VM installation.
- Read carefully the comments in the scripts. They show useful information on how to use them to log their output and their errors to different files and also what certain commands do and the reasons for their presence.
- Search for all types of possible errors in the log files to solve them after running the scripts.
 - Useful keywords: error, failed to build, etc.
- Initialize the data at the beginning of the scripts before you run them.
- It can take around 3 hours for the software-install script to be completed. When approximately 2 hours are elapsed it will ask you to confirm some install options (the Foxit Reader installation wizard and the addition of the Spanish DNle security device).
- The first usage on the software-install script allows you to log both the standard and the error output to a single file, while the second usage allows you to log each output to a different file.
- Shared folders will be located in `/mnt/hgfs` directory.
- The troubleshooting-install script must be run after a restart, otherwise it won't work.
- It is recommended to deactivate the sleep mode and the lock screen options to avoid problems during the installation process.
- Sometimes the names of the packages can change or they are not available anymore (those from AUR and those from the official repositories as well). Therefore, it is recommended to test the scripts on a virtual machine first and analyze deeply all the logs to check whether there are errors or not.
- Initialize the `nvidia_drivers` variable with `true` or with `"true"` to install the drivers required by Nvidia GPUs. If you have a different graphics card than me, the driver that I am installing may not be appropriate for you, therefore, I suggest you to investigate what driver you need exactly depending on what graphics card you have and then modify the Nvidia drivers section according to your needs. If you don't want to install these drivers, initialize the variable with `false` or with `"false"` .
- To avoid time issues in dual boot machines, you have two different options:
 - Set the hardware clock to be in local time instead of UTC on Arch Linux `sudo timedatectl set-local-rtc 1` and do nothing on Windows.
 - Set the hardware clock to be in UTC instead of local time on Arch Linux `sudo timedatectl set-local-rtc 0` and force Windows to treat the hardware clock as UTC instead of local time by modifying the registry `\HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\TimeZoneInformation\RealTimeIsUniversal=1` . This option is preferable because in this way Arch Linux will not have issues with daylight saving time or time zone changes.
- Grub OS-prober is disabled, that is why the `21_custom` and `25_custom` scripts have been created. These scripts personalize Debian Linux and Microsoft Windows meny entries. It is worth mentioning that you can do the same without disabling grub OS prober by editing `/boot/grub/grub.cfg` file, however take into account that this file will be overwritten on every grub update that you perform, therefore it will delete your modifications.
- There is a script that synchronizes the configured time zone of the machine with the time zone recognized by your internet connection. This script takes the time zone taking into account the location of your IP address, therefore if you are connected to a corporate network, hotspot or VPN it won't work properly. The script avoids trusting the time zone received if you are using a VPN or a hotspot, however the way to recognize a hotspot connection is not 100 % accurate (I have seen that at least the Android phones that I have tried, mark their connection as metered so a hotspot connection is recognized, but IOS doesn't so for Apple devices this doesn't work), therefore if you use sometimes hotspot networks check this before deciding to automate the time zone synchronization. If you use corporate networks frequently don't activate this option.

Commands to be run

```
setfont ter-128b

loadkeys es

ls /sys/firmware/efi/efivars

#####In case we have a wireless connection
iwctl

device list

station wlanX get-networks

station wlanX connect wifi_name

exit
#####In case we have a wireless connection

timedatectl set-ntp true

fdisk -l
connected to the system

fdisk /dev/sdX -l

cfdisk /dev/sdX

fdisk /dev/sdX -l

#####Only in case there is no other operating system already installed
mkfs.fat -F32 /dev/sdX1
#####Only in case there is no other operating system already installed

mkfs.ext4 /dev/sdX2

mkfs.ext4 /dev/sdX3

mkswap /dev/sdX4

mount /dev/sdX2 /mnt

mkdir /mnt/home && mount /dev/sdX3 /mnt/home
partition

swapon /dev/sdX4

cp /etc/pacman.d/mirrorlist /etc/pacman.d/mirrorlist.bak

reflector --country ES,PT,FR,GB,DE --age 12 --protocol https
12 hours located in close countries.

reflector --country ES,PT,FR,GB,DE --age 12 --protocol https --save /etc/pacman.d/mirrorlist

pacman -Syy
list

pacstrap /mnt base base-devel linux linux-headers linux-lts linux-lts-headers linux-firmware sudo nano vim ntfs-3g networkmanager git wget zsh
wezterm dolphin      #Install the Arch Linux system with a few useful tools

genfstab -U /mnt >> /mnt/etc/fstab

arch-chroot /mnt

cd tmp

git clone https://github.com/Tony450/AutoArchInstaller

cd AutoArchInstaller

chmod +x base-install.sh

./base-install.sh |& tee base-install.log

exit

umount -R /mnt

reboot

cd Downloads

git clone https://github.com/Tony450/AutoArchInstaller

cd AutoArchInstaller

chmod +x software-install.sh troubleshooting-install.log

./software-install.sh > >(tee software-install-stdout.log) 2> >(tee software-install-stderr.log >&2)

reboot

./troubleshooting-install.sh |& tee troubleshooting-install.log
```

```
#Set font

#Set keyboard layout

#Verify efi mode

#Start an interactive prompt

#See the list of available wireless adapters

#Print out a list of all the nearby networks

#Connect to the wifi_name network

#Update the system clock

#List all the partition tables of every device

#List all the partitions of sdX device

#Start the program to create new partitions

#Take a look at the recently created partitions

#Format the EFI partition

#Format the EXT4 partition (root)

#Format the EXT4 partition (home)

#Format the swap partition

#Mount the root partition

#Create the home directory and mount the home

#Tell Linux to use this partition as swap explicitly

#Make a backup of the default mirror list

#List mirrors that were synchronized within the last

#Update the default mirror list

#Update the package cache according to the new mirror list

#Generate the fstab file

#Login to the newly installed system

#Come out of the Arch-Chroot environment

#Unmount the partitions

#Reboot the machine
```

License

Copyright (C) 2024 Antonio Manuel Hernández De León.

Licensed under the [GPL license](#).

Conky theme made by [jxai](#) from <https://github.com/jxai/lean-conky-config>.

Grub theme made by [Hashir Sajid](#) from <https://github.com/hashirsajid58200p/earth-and-moon-grub-theme>.